

EventsApp HTTPS & Network Encryption (Milestone 5)

Owen Lindsey

Professor Sluiter, Shad

CST-407

Grand Canyon University

Overview

Enable SSL/TLS for the EventsApp so credentials are no longer sent in clear text. This milestone documents the before/after Wireshark captures, the creation of a self-signed certificate, and the Spring Boot configuration that enforces HTTPS with automatic HTTP → HTTPS redirection. Target project: `coding/CST-407-RS-T5-Milestone-Eventsapp/eventsapp` (MariaDB/MySQL backend).

Video Demonstration

Link (≤5 minutes): https://youtu.be/wHLpP_ZTLoQ

Walkthrough: (1) capture clear-text login over HTTP, (2) generate and configure a self-signed PKCS12 keystore, (3) force HTTPS with redirect + secure channel, (4) recapture the login to show encrypted payloads.

1. Baseline: Clear-Text Capture

1. Run the app on HTTP (port 8080) with no SSL properties set.
2. In Wireshark, filter on `http && tcp.port==8080`.
3. Submit the login form. Observe username/password visible in the payload. Save a screenshot for the “before” section.

Capture	Filter	Expected (before)
Login over HTTP	<code>http && tcp.port==8080</code>	Credentials visible in clear text

2. Create a Self-Signed Certificate

Generate a PKCS12 keystore in resources (already done; adjust passwords if desired):

```
cd coding/CST-407-RS-T5-Milestone-Eventsapp/eventsapp
keytool -genkeypair -alias eventsapp -keyalg RSA -keysize 2048 -storetype
PKCS12 \
  -keystore src/main/resources/keystore.p12 -validity 3650 \
  -storepass changeit -keypass changeit \
  -dname "CN=localhost, OU=Events, O=Events, L=Local, S=State, C=US"
```

3. Configure HTTPS + Redirect

application.properties

Add SSL properties and move the app to 8443 (HTTP listener for redirect on 8080):

```
server.port=8443
server.http.port=8080
server.ssl.enabled=true
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=changeit
server.ssl.key-store-type=PKCS12
server.ssl.key-alias=eventsapp
server.servlet.session.cookie.secure=true
```

HTTP → HTTPS Redirect (Tomcat)

Active config (adds HTTP connector + enforces CONFIDENTIAL on all URLs):

```
// src/main/java/com/shadsluiter/eventsapp/config/HttpsRedirectConfig.java
package com.shadsluiter.eventsapp.config;

import org.apache.catalina.connector.Connector;
import org.apache.catalina.core.StandardContext;
import org.apache.tomcat.util.descriptor.web.SecurityCollection;
import org.apache.tomcat.util.descriptor.web.SecurityConstraint;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.web.embedded.tomcat.TomcatServletWebServerFactory;
import org.springframework.boot.web.servlet.server.ServletWebServerFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class HttpsRedirectConfig {

    @Value("${server.http.port:8080}")
    private int httpPort;

    @Value("${server.port:8443}")
    private int httpsPort;

    @Bean
```

```

    public ServletWebServerFactory servletContainer() {
        TomcatServletWebServerFactory tomcat = new
TomcatServletWebServerFactory();
        tomcat.addAdditionalTomcatConnectors(createHttpConnector());
        tomcat.addContextCustomizers(context -> {
            if (context instanceof StandardContext standardContext) {
                SecurityConstraint constraint = new SecurityConstraint();
                constraint.setUserConstraint("CONFIDENTIAL");
                SecurityCollection collection = new SecurityCollection();
                collection.addPattern("/*");
                constraint.addCollection(collection);
                standardContext.addConstraint(constraint);
            }
        });
        return tomcat;
    }

    private Connector createHttpConnector() {
        Connector connector = new
Connector(TomcatServletWebServerFactory.DEFAULT_PROTOCOL);
        connector.setScheme("http");
        connector.setPort(httpPort);
        connector.setSecure(false);
        connector.setRedirectPort(httpsPort);
        return connector;
    }
}

```

4. Verify Encrypted Communication

1. Restart the app. Browse to `http://localhost:8080` to see the redirect, then use `https://localhost:8443` (accept the self-signed warning).
2. Capture with tshark/Wireshark on TLS: `sudo tshark -i lo -f 'tcp port 8443' -w /tmp/capture-https.pcapng` then `tshark -r /tmp/capture-https.pcapng -Y tls -o tls.`
3. Submit the login form. Payload is encrypted; no `http.file_data` visible.
4. For “before” proof: `sudo tshark -i lo -f 'tcp port 8080' -w /tmp/capture-http.pcapng` then `tshark -r /tmp/capture-http.pcapng -Y 'http.request.method == "POST"' -o http and decode body with ... -T fields -e http.file_data | xxd -r -p.`

Capture	Filter	Expected (after)
Login over HTTPS	<code>tcp.port==8443 or tls</code>	Encrypted TLS records; no readable credentials

Why HTTPS Matters (what we fixed)

Moving to TLS removes clear-text credentials and blocks downgrade attempts. Summary of before/after:

Aspect	HTTP (before)	HTTPS (after)
Credentials on the wire	Visible in <code>http.file_data</code>	Encrypted (not visible)
Port	8080	8443 (8080 now only redirects)
Wireshark view	HTTP POST with body shown	TLS handshake + encrypted Application Data
User trust	No cert	Self-signed (browser warning is expected in lab)

Runtime Behavior Changes

1. The app listens on 8443 (HTTPS) and 8080 (HTTP redirect only).
2. Tomcat enforces `CONFIDENTIAL`, so all URLs require a secure channel.
3. Session cookies are marked secure (`server.servlet.session.cookie.secure=true`) to avoid HTTP leakage.

Evidence Summary (for the report/video)

View	Command	Expected evidence
HTTP (before)	<code>tshark -r /tmp/capture-http.pcapng -Y 'http.request.method == "POST"' -T fields -e http.file_data \ xxd -r -p</code>	Shows <code>username=...&password=...</code>
HTTPS (after)	<code>tshark -r /tmp/capture-https.pcapng -Y tls -0 tls</code>	Only TLS records; no <code>http.file_data</code>
Redirect check	Hit <code>http://localhost:8080</code> in browser/curl	302 to <code>https://localhost:8443</code>

Database Proof (optional for narrative)

Passwords remain hashed in the database (`SELECT id, login_name, password FROM users;`). The clear-text you captured over HTTP illustrates transport risk, not storage weakness; hashes (bcrypt) are expected and correct.

```

~ ) sudo tshark -i lo -f 'tcp port 8080' -w /tmp/capture-http.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
12 ^C

~ ) sudo tshark -r /tmp/capture-http.pcapng -Y 'http.request.method == "POST"' -O http
Running as user "root" and group "root". This could be dangerous.
Frame 1: Packet, 955 bytes on wire (7640 bits), 955 bytes captured (7640 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 6, Src: ::1, Dst: ::1
Transmission Control Protocol, Src Port: 50674, Dst Port: 8080, Seq: 1, Ack: 1, Len: 869
Hypertext Transfer Protocol
  POST /users/login HTTP/1.1\r\n
    Request Method: POST
    Request URI: /users/login
    Request Version: HTTP/1.1
  Host: localhost:8080\r\n
  Connection: keep-alive\r\n
  Content-Length: 37\r\n
    [Content length: 37]
  Cache-Control: max-age=0\r\n
  sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"\r\n
  sec-ch-ua-mobile: ?0\r\n
  sec-ch-ua-platform: "Linux"\r\n
  Origin: http://localhost:8080\r\n
  Content-Type: application/x-www-form-urlencoded\r\n
  Upgrade-Insecure-Requests: 1\r\n
  User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36\r\n
  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
  Sec-Fetch-Site: same-origin\r\n
  Sec-Fetch-Mode: navigate\r\n
  Sec-Fetch-User: ?1\r\n
  Sec-Fetch-Dest: document\r\n
  Referer: http://localhost:8080/users/loginForm?logout=true\r\n
  Accept-Encoding: gzip, deflate, br, zstd\r\n
  Accept-Language: en-US,en;q=0.9\r\n
  \r\n
  [Full request URI: http://localhost:8080/users/login]
  File Data: 37 bytes
HTML Form URL Encoded: application/x-www-form-urlencoded

~ ) sudo tshark -r /tmp/capture-http.pcapng -Y 'http.request.method == "POST"' -T fields -e http.file_data | xxd -r -p
Running as user "root" and group "root". This could be dangerous.
username=Owen+Lindsey&password=badguy
~ ) mysql -u root -proot -D eventsapp -e "SELECT id, login_name, password FROM users;"
mysql: Deprecated program name. It will be removed in a future release, use '/usr/bin/mariadb' instead
+-----+-----+-----+
| id | login_name | password |
+-----+-----+-----+
| 1 | root | root |
| 2 | omni | $2a$10$kZ//kb6KMiWbp8z1ITWbqu5hjfq2wy4wy4LLjQrLAmXH4iYxJUQsu |
| 3 | omniv | $2a$10$i4cD0f3I70LzrnD0AX4CL.e7UEETeCWfVuTPf6Fd4vS85znHnr.D2 |
| 4 | roger | $2a$10$Uf8274P6V3s6QJu0L3Xm3u.zHaZjPF01ENXvqb6NSQvwZIOfIHv4. |
| 5 | Owen Lindsey | $2a$10$PjxBpSooYoAvy9/40zDpn0m/9zo0Mv21f98P8XTwew5HCCr6w154. |
+-----+-----+-----+

~ ) mysql -u root -proot -D eventsapp -e "SELECT id, login_name, password FROM users WHERE login_name='Owen Lindsey'\6"
mysql: Deprecated program name. It will be removed in a future release, use '/usr/bin/mariadb' instead
***** 1 row *****

```

```
id: 5
login_name: Owen Lindsey
password: $2a$10$P1xRpSooYoAvv9/40zDnn0m/9zo0My21f98P8XTwew5HCCr6w154.
```

Trusting the Self-Signed Cert (browser note)

1. Expect a browser warning on first visit to `https://localhost:8443` ; accept for the lab.
 2. To remove the warning, import `keystore.p12` into your OS/browser trust store or replace with a CA-signed certificate.
-

5. Source Code / Config Changes

- `src/main/resources/application.properties` – SSL keystore config (`keystore.p12`), HTTPS port 8443, HTTP redirect port 8080.
 - `src/main/java/com/shadsluiter/eventsapp/config/HttpsRedirectConfig.java` – Adds HTTP connector and enforces `CONFIDENTIAL` (auto-redirect).
 - Keystore: `src/main/resources/keystore.p12` (self-signed, password `changeit` , alias `eventsapp`).
 - `SecurityConfig` unchanged; redirect is enforced via Tomcat security constraint.
-

6. How to Run

```
cd coding/CST-407-RS-T5-Milestone-Eventsapp/eventsapp
mvn spring-boot:run
```

Browse to `https://localhost:8443` . HTTP on `http://localhost:8080` should redirect to HTTPS. Update `application.properties` if you change the keystore path or passwords. Accept the self-signed cert in your browser for the demo.